



UNIVERSITÉ DE RENNES
MASTER MATHÉMATIQUES ET APPLICATIONS
PARCOURS MATHÉMATIQUES DE L'INFORMATION, CRYPTOGRAPHIE

Rapport d'Implémentation

Moteur cryptographique AES et chiffrement authentifié GCM

Alexandre ACCIARI

Professeur : Sylvain Duquesne

8 mai 2026

Résumé

Ce rapport détaille l'implémentation complète et à partir de zéro de l'algorithme de chiffrement AES (128, 192 et 256 bits) en langage C. Il présente l'architecture du code, les solutions mathématiques apportées pour les calculs dans le corps de Galois, ainsi que la gestion rigoureuse des fichiers via différents modes opératoires (ECB, CBC, CFB, OFB). Le projet possède aussi l'implémentation du chiffrement et du déchiffrement authentifié GCM (Galois/Counter Mode), assurant à la fois confidentialité et intégrité des données. Enfin, une analyse des difficultés / solutions perçus mettant en évidence l'exigence du développement cryptographique dans chacune de ces parties.

Table des matières

1	Introduction	3
2	Architecture du Code	3
3	Le Moteur AES et l'Arithmétique des Corps Finis	3
3.1	Flexibilité des clés et Key Expansion	4
3.2	Arithmétique dans le Corps de Galois $GF(2^8)$	4
4	Modes Opératoires Classiques et Traitement des Fichiers	4
4.1	De la vulnérabilité de l'ECB à la sécurité du CBC	4
4.2	Le standard de remplissage PKCS #7	5
5	Le Boss Final : Le Mode GCM (AEAD)	5
5.1	Confidentialité : Le moteur GCTR	5
5.2	Authentification : Le hachage universel GHASH	5
6	Défis Techniques et Solutions Logicielles	5
6.1	Sécurité des entrées/sorties et gestion des flux	6
6.2	Le problème de Boutisme (Endianness)	6
6.3	Gestion de la mémoire et padding virtuel GCM	6

1 Introduction

Dans le cadre de ce projet, nous avons développé un moteur de chiffrement et de déchiffrement basé sur le standard **Advanced Encryption Standard (AES)**. L'objectif principal était de comprendre et d'implémenter de zéro les rouages mathématiques complexes de cet algorithme symétrique, tel que défini par la norme **FIPS-197** [1].

Au-delà de l'algorithme de base, ce projet nécessitait de concevoir une application console, capable de traiter des fichiers de taille arbitraire. Cela a impliqué le développement de plusieurs modes opératoires classiques (**ECB**, **CBC**, **CFB**, **OFB**). Étant donné que l'algorithme AES traite les données par blocs stricts de 16 octets, la gestion des fins de fichiers a nécessité l'implémentation rigoureuse du standard de remplissage **PKCS #7** (défini par l'IETF dans la RFC 2315 [2]), garantissant ainsi une restitution des données lors du déchiffrement. Aussi, pour répondre aux standards de sécurité les plus récents, l'application a finalement été étendue pour intégrer le mode **GCM (Galois/Counter Mode)**, offrant un chiffrement authentifié avec données associées (AEAD) conformément à la spécification **NIST SP 800-38D**[3].

2 Architecture du Code

Afin de garantir un code propre en suivant les bonnes pratiques de développement en C, le projet a été strictement divisé en plusieurs composants :

- **Le cœur cryptographique (src/aes.c et include/aes.h)** : Ce module isole toute la logique mathématique de l'AES. Il contient l'expansion de clé (KeyExpansion), les transformations d'état (SubBytes, ShiftRows, MixColumns, AddRoundKey) ainsi que l'arithmétique spécifique aux modes GCM.
- **L'interface utilisateur et la gestion des I/O (src/main.c)** : Ce composant constitue le point d'entrée du programme. Il gère le parsing des arguments via `getopt_long`, l'allocation dynamique de la mémoire, le découpage des flux en blocs itératifs, l'application du padding PKCS #7, et la validation cryptographique finale des tags GCM.
- **L'automatisation (Makefile)** : Un système de compilation automatisé a été mis en place. L'utilisation de drapeaux de compilation stricts (`-Wall -Wextra`) a permis de sécuriser le code contre les comportements indéfinis, tandis que le drapeau `-O3` a été crucial pour optimiser les performances des nombreuses boucles mathématiques lors de l'exécution.

3 Le Moteur AES et l'Arithmétique des Corps Finis

L'algorithme AES opère sur une matrice d'état (*State*) de 4 lignes et 4 colonnes, représentant un bloc de 16 octets.

3.1 Flexibilité des clés et Key Expansion

Pour répondre aux besoins de sécurité variables, notre implémentation supporte dynamiquement les clés de 128, 192 et 256 bits. La fonction d'expansion de clé s'adapte mathématiquement à la taille fournie ($N_k = 4, 6$ ou 8 mots de 32 bits) pour déduire le nombre de tours requis ($N_r = 10, 12$ ou 14). Remarque, une attention particulière a été portée à la spécificité mathématique de l'AES-256 : lorsque $N_k > 6$, l'algorithme exige une étape de substitution non linéaire supplémentaire (**SubWord**) au milieu de la génération des clés de tours pour éviter l'effritement de l'entropie [1].

3.2 Arithmétique dans le Corps de Galois $GF(2^8)$

L'une des difficultés majeures de l'implémentation logicielle réside dans l'étape **MixColumns**. Cette transformation nécessite des multiplications matricielles qui ne s'effectuent pas dans l'arithmétique standard, mais dans le corps fini de Galois $GF(2^8)$, défini par le polynôme irréductible $P(x) = x^8 + x^4 + x^3 + x + 1$.

Pour transposer ces calculs en langage machine de manière performante, nous avons développé la fonction fondamentale **xtime**. Cette fonction effectue une multiplication par le polynôme x (soit la valeur hexadécimale **0x02**) via un décalage binaire vers la gauche ($\ll 1$), suivi d'un **XOR** conditionnel avec la constante **0x1B** si un dépassement de capacité (débordement du bit de poids fort) survient. Les multiplications par des scalaires plus complexes, indispensables notamment pour l'opération **InvMixColumns** lors du déchiffrement (multiplications par **0x09**, **0x0B**, **0x0D** et **0x0E**), ont été construites par un algorithme de type *shift-and-add* basé sur **xtime**.

4 Modes Opérateurs Classiques et Traitement des Fichiers

L'AES étant un algorithme de chiffrement par blocs, la sécurisation d'un fichier de taille arbitraire nécessite un "Mode d'Opération".

4.1 De la vulnérabilité de l'ECB à la sécurité du CBC

Nous avons initialement implémenté le mode **ECB** (*Electronic Codebook*), qui chiffre chaque bloc indépendamment. Cependant, ce mode présente un défaut cryptographique majeur : deux blocs de texte clair identiques produiront un texte chiffré identique, préservant ainsi les structures visuelles (comme l'illustre classiquement le chiffrement d'une image bitmap non compressée).

Pour pallier ce défaut, nous avons intégré le mode **CBC** (*Cipher Block Chaining*). Ce mode introduit un *Vecteur d'Initialisation* (*IV*) fourni par l'utilisateur. Chaque bloc de texte clair subit un XOR avec le bloc chiffré précédent avant d'entrer dans l'AES. Cette chaîne de dépendance garantit que des blocs clairs identiques produiront des chiffrés totalement différents, offrant une diffusion parfaite. Les modes **CFB** et **OFB** ont également été codés pour offrir des approches par chiffrement de flux (*stream ciphers*), où l'AES est utilisé comme générateur de nombres pseudo-aléatoires.

4.2 Le standard de remplissage PKCS #7

Dans les modes opératoires par blocs (ECB, CBC), la taille du fichier d'entrée n'est que très rarement un multiple exact de 16 octets. Le document NIST et la RFC 2315 [2] proposent un mécanisme de *padding* : si le dernier bloc contient N octets de données (avec $N < 16$), il doit être complété par $(16 - N)$ octets ayant tous pour valeur $(16 - N)$.

Le piège de la fin de fichier parfaite :

Lors de nos phases de tests, nous avons mis en évidence la nécessité absolue d'appliquer la règle du bloc supplémentaire forcé lorsque le fichier source est un multiple exact de 16 octets. Pour illustrer cette contrainte, nous avons tenté de chiffrer la chaîne de caractères "*Cryptomonnaie!*\n". Cette chaîne, avec son caractère de saut de ligne final (\n), pèse très exactement 16 octets.

Sans l'ajout d'un nouveau bloc complet de remplissage (constitué de 16 octets de valeur 0x10), l'algorithme de déchiffrement va lire le dernier octet de la chaîne chiffrée pour en déduire le *padding*. Or, le caractère \n correspond à la valeur décimale 10 en ASCII. Le déchiffreur ampute alors silencieusement le message de 10 octets par erreur, restituant ainsi le mot tronqué "*Crypto*". Cette observation empirique valide de manière frappante l'exigence du standard PKCS #7 de toujours fournir une indication non ambiguë de la fin du fichier en forçant un bloc complet si nécessaire.

5 Le Boss Final : Le Mode GCM (AEAD)

Le projet s'est conclu par l'implémentation du mode **GCM (Galois/Counter Mode)**, le standard actuel utilisé dans les protocoles sécurisés comme TLS 1.3 [3]. Ce mode repose sur deux piliers :

5.1 Confidentialité : Le moteur GCTR

La confidentialité est assurée par une variante du mode Counter (CTR). Un compteur de 128 bits, initialisé par un IV (recommandé à 96 bits), est chiffré par l'AES pour créer un flux de clés. L'avantage majeur de ce mode par flot est de rendre le padding PKCS #7 obsolète : les données n'ont plus besoin d'être alignées sur 16 octets. Le dernier bloc partiel est chiffré par un simple XOR tronqué, optimisant ainsi la taille du fichier de sortie.

5.2 Authentification : Le hachage universel GHASH

L'intégrité est assurée par la fonction GHASH, qui absorbe le texte chiffré et produit un *Tag* d'authentification de 16 octets. L'opération repose sur la multiplication de polynômes dans l'immense corps $GF(2^{128})$. Au lieu du polynôme réducteur classique de l'AES, GCM utilise $R = 11100001 || 0^{120}$ (0xE1...). L'implémentation de `gcm_mult` effectue cette réduction bit par bit, en gérant les décalages croisés de l'ensemble des 128 bits du bloc.

6 Défis Techniques et Solutions Logicielles

La traduction de protocoles cryptographiques de très haut niveau en langage C soulève de nombreux défis architecturaux.

6.1 Sécurité des entrées/sorties et gestion des flux

Lors du développement initial, l'utilisation de la fonction standard `fread()` levait des avertissements de compilation (`warn_unused_result`). Plus grave encore, une implémentation naïve avait conduit à un bug logique de "double lecture" du dernier bloc de fichier lors de l'extraction du Tag GCM. La solution retenue a consisté à encapsuler rigoureusement chaque Input/Output dans des structures conditionnelles, en s'assurant que la variable `bytes_read` correspondait exactement à la taille attendue, neutralisant ainsi les comportements indéfinis avant tout traitement cryptographique.

6.2 Le problème de Boutisme (Endianness)

L'algorithme GCM requiert que la taille totale des données soit encodée sur 64 bits en format **Big-Endian** pour finaliser le dernier bloc du GHASH. Nos processeurs x86 travaillant nativement en **Little-Endian**, une copie mémoire classique (`memcpy`) inversait l'ordre des octets, provoquant systématiquement la corruption du Tag final. Nous avons résolu ce problème en concevant une fonction (`put_uint64_be`) utilisant des masques logiques (`& 0xFF`) et des décalages binaires (`>> 8`) pour forcer l'ordre naturel des octets indépendamment de l'architecture matérielle de la machine.

6.3 Gestion de la mémoire et padding virtuel GCM

Enfin, le NIST exige que les données chiffrées entrant dans le GHASH soient concaténées avec des blocs de zéros virtuels (padding mathématique 0^u et 0^v) [3]. Insérer manuellement ces zéros via des boucles dans des tableaux dynamiques risquait de provoquer des fuites ou des dépassements mémoire. La solution élégante a été d'utiliser l'allocation `calloc()` native du C, qui initialise d'emblée un grand tampon avec des zéros. En gérant judicieusement un curseur d'écriture (*offset*), les données y ont été superposées, réalisant le padding de manière instantanée et sécurisée.

Bibliographie

- [1] National Institute of Standards and Technology (NIST), *Advanced Encryption Standard (AES)*, Federal Information Processing Standards Publication (FIPS PUB) 197, Novembre 2001.
- [2] B. Kaliski (RSA Laboratories), *PKCS #7 : Cryptographic Message Syntax Version 1.5*, Request for Comments (RFC) 2315, Network Working Group, Mars 1998.
- [3] M. Dworkin (NIST), *Recommendation for Block Cipher Modes of Operation : Galois/Counter Mode (GCM) and GMAC*, NIST Special Publication 800-38D, Novembre 2007.